

오늘의 작업기록 — 2026년 7월 22일

복기지 구축 — 작업기록이 스스로 문제은행이 되는 파이프라인

(수율이 떨어진 원인을 진단하고, 그 처방을 도구로 만든 날)

[문서 규칙] 발체 없이 풀코드·풀출력으로 기재한다. 오늘 만든 두 파일은 전문을 싣는다 — 6개월 뒤 이 파일만으로 재현 가능해야 하므로.

한눈에

항목	내용
동기	"IDEA 때보다 머리에 남는 게 적다" — 폰 워크플로우가 복불 위주라 생성형 타이핑이 빠짐
만든 것	make_review.py (로그 → 문제은행 추출) + review.html (정적 복기 페이지)
배선	크론을 make_review.py ; update.py 로 교체 → 작업기록 올리면 1분 내 문제 자동 갱신
주소	taewoo-won.github.io/backend-journey/review.html — 비로그인 공개 (사지방 브라우저용)
두 모드	문제 복기 (날짜/★/랜덤 필터 + 답안 타이핑 + O·△·X 자가채점) · 코드 재현 (독타이핑 → diff 채점 + 타이머)
★ 설계 판단	AI를 페이지에 안 넣는다 — 공개·비로그인이라 키 노출. 대신 [답안 내보내기]→네이버 내게쓰기→채팅으로 비동기 채점
★ 핵심 발견	문제를 " 생성 "이 아니라 " 수거 " — 검증 질문 섹션 + C축 풀코드가 이미 매일 문제은행을 만들고 있었다
배포 마찰 2	① 폰 SFTP가 파일명 언더바를 공백으로 바꿈 ② base64 한 줄 2376자가 폰에서 잘림 → 400자 3조각 분할로 해결
최종 검증	days: 32 문항: 163 코드: 57 · Pages review.html: 200 review-data.json: 200
잔디	크론 자동 커밋 9f6149c update: 2026-07-22

오늘 만든 건 학습 그 자체가 아니라 **학습의 인프라**다. 근데 이게 앞으로 매일의 수율을 바꾸는 거라, 하루 쓸 값어치가 충분했다. 그리고 만들면서 배운 것(결정적 출력, 원자적 교체, ; 와 && 의 차이, LCS 이전의 순진한 비교)도 그 자체로 백엔드 소재였다.

0. 동기 — 수율이 왜 떨어졌나

체감부터 정직하게: **IDEA로 하던 시절보다 머리에 남는 게 적다**. 이걸 "폰이라서"로 넘기면 해결이 안 되니 원인을 쫓겠다.

시기	손이 하는 일	인지적으로 일어나는 것
입대 전(IDEA)	코드를 생성 한다 — 빈 화면에서 짜고, 오타 내고, 고침	인출 + 생성 → 깊은 인코딩
지금(폰 Termius)	명령을 옮긴다 — 받은 걸 붙여넣고 결과를 확인	인식(recognition) 위주 → 얕은 인코딩

즉 문제는 **화면 크기가 아니라 손의 역할**이다. 복불 위주 워크플로우에선 "읽으면 아는데 쓰려면 못 쓰는" 상태가 쌓인다. 실제 증거도 이미 있었다 — 7/14에 **IllegalArgumentException** 이름이 안 나왔고(6/25에 내가 직접 내린 결정인데도), record를 두 번째 쓰면서도 필드 하나를 빠뜨렸다(7/15). 지식이 없는 게 아니라 **인출 경**로가 약한 것.

처방의 조건 세 개: 1. **사지방에서 돌아야 한다** — 부대 PC는 GitHub 로그인인 안 되고(7/15 확인), 개발환경 설치도 못 한다. **브라우저만으로** 되는 것이어야 한다. 2. **생성형 타이핑을 되돌려야 한다** — 읽고 고르는 게 아니라 빈 화면에 직접 쳐야 한다. 3. **손이 더 가면 안 된다** — 문제를 따로 만들어야 하면 며칠 만에 그만둔다. 이미 하는 일(작업기록)에 얹혀야 한다.

1. ★ 핵심 발견 — 인프라가 이미 90% 서 있었다

조건 3번을 고민하다 깨달은 것: **내 작업기록 포맷이 이미 이걸 위해 설계돼 있었다**.

이미 매일 생산되던 것	원래 목적	복기지에서의 역할
"검증 질문 (답 적지 말 것)" 섹션	다음 복기용 문항을 그날 박아두기	그대로 문제은행
C축 원칙의 풀코드 블록	파일 하나로 재현 가능하게	독타이핑 원본
backend-journey = 공개 GitHub Pages	포폴 대시보드	비로그인 접근 경로
매분 도는 크론(7/20에 정제 확인)	로그·로드맵 자동 동기화	자동 갱신 엔진

그래서 설계가 뒤집혔다 — 문제를 "**생성**"할 필요가 없다. "**수거**"만 하면 된다.

이게 왜 중요한가: AI가 문제를 지어내는 방식이었다면 ① 매번 호출 비용 ② 내 맥락을 모르는 일반적 문제 ③ 그날 실제로 배운 것과 어긋날 위험이 있다. 반면 수거 방식은 **내가 그날 스스로 "이건 물어봐야 한다"고 판단해 박아둔 문항**이 그대로 나온다. 문제의 질이 곧 작업기록의 질이고, 그건 이미 관리되고 있다.

연료 조건 하나: 검증 질문 섹션을 계속 쓰는 것. 그제 빠진 날은 그날 문항이 0이 된다.

2. 설계 판단 — AI를 어디에 둘 것인가

가장 오래 고민한 지점. "문제 푸는 과정에 AI가 안 들어가는데 지장 없냐"는 물음이 정확했다. 세 안을 놓고 비교했다.

안	방식	평가
① 페이지에 Claude API 직접	review.html에서 fetch로 채점 요청	기각. 공개·비로그인 페이지라 API 카를 넣으면 그대로 노출된다. 감추려면 VPS에 프록시 서버를 세워야 하는데 비용+보안면+복잡도가 다 늘어난다
② 비동기 루프 (채택)	답안 내보내기 → 네이버 내게쓰기 → 채팅에 붙여 채점	채택. 키 노출 0, 서버 0. 게다가 채점하는 쪽이 7주 이력을 다 아는 상태 라 컨텍스트 없는 API 한 방보다 채점 질이 높다
③ 자가채점 (페이지 내장)	출처 로그를 읽고 O·△·X 셀프 판정	병행 채택. Anki 등에서 검증된 방식. 즉시 피드백이 필요한 대다수 문항은 이걸로 충분

결론: **v1 = ③ + ②**. 사지방에선 ③으로 즉시 돌리고, △·X만 ②로 가져온다. 즉 **AI가 빠진 게 아니라 비동기로 들어간다**.

모드별로 갈리는 것도 정리했다: - 코드 재현 → AI 불필요. 원본과의 대조는 기계가 하는 객관 채점이라 오히려 정확하다. 그리고 수율 문제의 본체(생성형 타이핑 부족)를 정면으로 푸는 게 이쪽이다. - 서술형 → 쓰는 것 자체가 이미 인출 훈련. 수율의 대부분은 "쓰는 순간"에 나오지 채점에서 나오는 게 아니다. 약한 고리는 피드백인데 그건 ②·③이 메운다.

한게 하나 정직하게: 검증 질문은 "답 적지 말 것" 원칙이라 로그에 모범답안이 없다. 그래서 ③의 [출처 열기]가 정답지 역할을 한다 — 그 질문이 나온 섹션을 다시 읽는 것 자체가 목적 있는 재독이 된다.

3. make_review.py — 로그를 문제은행으로 (플코드)

설계 원본 (주석 포함 버전)

```
#!/usr/bin/env python3
"""logs/*.md -> review-data.json (복기지 데이터).

크론이 update.py '직전'에 매분 실행한다. 원칙 둘:
1) 절대 죽지 않는다 (파싱 실패한 파일은 건너뛰고, 전체 예외도 삼킨 뒤 exit 0)
   -> 이 스크립트가 죽어도 update.py(대시보드 봇)는 계속 돌아야 하므로.
2) 결정적 출력 (타임스탬프 없음, 정렬 고정)
   -> 로그가 안 바뀌면 JSON 바이트가 동일 -> git이 변경 없음으로 봐서 커밋 스팸 없음.

추출 대상:
- 각 날짜 파일의 "검증 질문" 섹션의 번호 문항 (★ 포함 여부 표시)
- ``java 코드 블록 (4줄 이상만; 직전 제목을 라벨로) -> 코드 재현(독타이핑) 소재
"""

import json
import os
import re
import sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
LOGS = HERE / "logs"
OUT = HERE / "review-data.json"

def parse_day(md: str, date: str) -> dict:
    # 제목: card의 title: 줄 우선, 없으면 첫 '## ' 제목
    title = ""

    m = re.search(r"^title:\s*(.+)$", md, re.M)
    if m:
        title = m.group(1).strip()
    else:
        m2 = re.search(r"^##\s+(.+)$", md, re.M)
        if m2:
            title = m2.group(1).strip()

    # 검증 질문 섹션: '## ... 검증 질문 ...' 부터 다음 '## ' 또는 '-----' 전까지
    questions = []

    qm = re.search(r"^##.*검증 질문.*$", md, re.M)
    if qm:
        seg = md[qm.end():]
        stop = re.search(r"^(##\s|-----)", seg, re.M)
        if stop:
            seg = seg[: stop.start()]

    return {
        "date": date,
        "title": title,
        "questions": questions
    }
```

```

        for n, q in re.findall(r"^(\\d+)\\.\\s+(.+)\\$", seg, re.M):
            q = q.strip()
            questions.append({"n": int(n), "q": q, "star": "★" in q})

# java 코드 블록 (4줄 미만 조각은 제외 - 재현 훈련 소재로 무의미)
codes = []
for cm in re.finditer(r"```java\\n(.*)```", md, re.S):
    code = cm.group(1).rstrip("\\n")
    if code.count("\\n") + 1 < 4:
        continue

    heads = re.findall(r"^#{2,3}\\s+(.+)\\$", md[: cm.start()], re.M)
    head = heads[-1].strip() if heads else "코드"
    codes.append({"title": head, "code": code})

return {"date": date, "title": title, "questions": questions, "codes": codes}

def main() -> None:
    days = []
    for p in sorted(LOGS.glob("2026-*.md")):
        try:
            days.append(parse_day(p.read_text(encoding="utf-8"), p.stem))
        except Exception:
            continue # 한 파일이 이상해도 나머지는 산다
    data = {"latest": days[-1]["date"] if days else "", "days": days}
    txt = json.dumps(data, ensure_ascii=False, separators=(",", ":"), sort_keys=True)
    if OUT.exists():
        try:
            if OUT.read_text(encoding="utf-8") == txt:
                return # 변경 없음 -> 쓰지 않음 (커밋 스펀 방지책의 핵심)
        except Exception:
            pass
    tmp = OUT.with_suffix(".json.tmp")
    tmp.write_text(txt, encoding="utf-8")
    os.replace(tmp, OUT) # 원자적 교체 - 반쯤 쓰인 JSON이 서빙되는 일 없음

if __name__ == "__main__":
    try:
        main()
    except Exception as e: # 어떤 일이 있어도 update.py를 막지 않는다
        print("make_review 오류(무시하고 계속):", e, file=sys.stderr)
    sys.exit(0)

```

실제 VPS에 배포된 버전 (주석 제거 압축판)

배포 마찰(§ 6) 때문에 base64 길이를 줄여야 해서, **동작은 동일하되 주석·공백을 걷어낸 판**이 실제로 올라갔다. 두 버전의 산출물이 같은지는 검증했다(둘 다 **days: 32 문항: 163 코드: 57**).

```

import json,os,re,sys
from pathlib import Path
H=Path(__file__).resolve().parent
def day(md,d):
    m=re.search(r"^title:\s*(.+)$",md,re.M) or re.search(r"^#\s+(.+)$",md,re.M)
    t=m.group(1).strip() if m else ""
    qs=[]
    qm=re.search(r"^##.*검증 질문.*$",md,re.M)
    if qm:
        s=md[qm.end():]
        st=re.search(r"^(#\s|-----)",s,re.M)
        if st:s=s[:st.start()]
        for n,q in re.findall(r"^(\d+)\.\s+(.+)$",s,re.M):
            q=q.strip();qs.append({"n":int(n),"q":q,"star":"★" in q})
    cs=[]
    for c in re.finditer(r"```java\n(?:.*?)```",md,re.S):
        code=c.group(1).rstrip("\n")
        if code.count("\n")+1<4:continue
        hs=re.findall(r"^#{2,3}\s+(.+)$",md[:c.start()],re.M)
        cs.append({"title":hs[-1].strip() if hs else "코드","code":code})
    return {"date":d,"title":t,"questions":qs,"codes":cs}
def main():
    ds=[]
    for p in sorted((H/"logs").glob("2026-*.md")):
        try:ds.append(day(p.read_text(encoding="utf-8"),p.stem))
        except Exception:continue
    o=H/"review-data.json"
    tx=json.dumps({"latest":ds[-1]["date"] if ds else "", "days":ds},ensure_ascii=False,separators=(",", ":"),sort_keys=True)
    if o.exists():
        try:
            if o.read_text(encoding="utf-8")==tx:return
        except Exception:pass
    tmp=o.with_suffix(".json.tmp");tmp.write_text(tx,encoding="utf-8");os.replace(tmp,o)
try:main()
except Exception as e:print("mr err:",e,file=sys.stderr)
sys.exit(0)

```

코드 해설 — 왜 이렇게 짰나

① 문항 추출 정규식

```
qm = re.search(r"^##.*검증 질문.*$", md, re.M)
```

re.M (MULTILINE)은 **^**와 **\$**를 줄 단위로 해석하게 한다. 이게 없으면 문서 전체의 시작·끝만 가리켜서 안 잡힌다. **##**로 시작하고 "검증 질문"을 포함하는 줄 = 그 섹션의 머리.

```
stop = re.search(r"^(#\s|-----)", seg, re.M)
```

섹션의 끝을 어디로 볼 것인가 — 다음 **##** 제목이 나오거나 **-----** 구분선이 나오면 거기까지. 내 작업기록 포맷이 항상 이 둘 중 하나로 섹션을 끊으니 안전하다.

```
for n, q in re.findall(r"^(\d+)\.\s+(.+)$", seg, re.M):
```

1. 질문내용 끝을 잡아 번호와 본문을 분리한다. 괄호 두 개가 캡처 그룹이라 (**번호**, **본문**) 튜플로 돌아온다.

```
questions.append({"n": int(n), "q": q, "star": "★" in q})
```

★이 붙은 문항은 그날의 핵심이라 표시해둔다 — 페이지에서 "★만" 필터로 쓴다.

② 코드 블록 추출

```
for cm in re.finditer(r"```java\n(?:.*?)```", md, re.S):
```

re.S (DOTALL)는 **.**이 줄바꿈까지 먹게 한다 — 여러 줄 코드 블록을 통째로 잡으려면 필수. **.+?**는 **비탐욕(non-greedy)** 매칭이라 가장 가까운 닫는 백틱까지만 먹는다. 탐욕(**.***)으로 하면 문서의 첫 코드블록 시작부터 마지막 코드블록 끝까지 한 덩어리로 잡혀버린다.

```
if code.count("\n") + 1 < 4:
    continue
```

4줄 미만은 버린다. 한두 줄짜리는 재현 훈련 소재로 무의미하고, 목록만 지저분해진다.

```
heads = re.findall(r"^(#{2,3}\s+(.+)$", md[: cm.start()], re.M)
head = heads[-1].strip() if heads else "코드"
```

코드 블록 앞부분 (`md[:cm.start()]`)에서 제목들을 다 찾은 뒤 마지막 것을 라벨로 쓴다 = 그 코드 바로 위의 제목. 그래서 목록에 "완성된 TransferController 전문" 같은 이름이 뜬다.

③ 결정적 출력 (커밋 스팸 방지의 핵심)

```
txt = json.dumps(data, ensure_ascii=False, separators=(",", ":"), sort_keys=True)
```

- `ensure_ascii=False` — 한글을 `\uXXXX` 로 안 바꾸고 그대로 쓴다(용량·가독성).
- `separators=(",", ":")` — 공백 없는 최소 형태.
- `sort_keys=True` — 키 순서를 고정. 이게 없으면 실행마다 순서가 달라져 파일이 매번 바뀔 것처럼 보인다.
- 타임스탬프를 넣지 않는다 — 넣는 순간 매분 파일이 달라지고, 크론이 매분 커밋하는 스팸이 된다.

```
if OUT.read_text(encoding="utf-8") == txt:
    return
```

기존 파일과 내용이 같으면 아예 쓰지 않는다. 파일 mtime조차 안 건드리니 git이 완전히 조용하다.

④ 원자적 교체

```
tmp = OUT.with_suffix(".json.tmp")
tmp.write_text(txt, encoding="utf-8")
os.replace(tmp, OUT)
```

임시 파일에 다 쓴 뒤 `os.replace` 로 갈아끼운다. 직접 덮어쓰면 쓰는 도중에 누가 읽을 때 반쯤 쓰인 깨진 JSON이 서빙될 수 있다. `os.replace` 는 OS 레벨에서 원자적이라 그 틈이 없다.

⑤ 절대 죽지 않기

```
try:
    main()
except Exception as e:
    print("make_review 오류(무시하고 계속):", e, file=sys.stderr)
sys.exit(0)
```

어떤 예외가 나도 `exit 0`로 끝난다. 이유는 § 5(크론 배선)에서.

4. review.html — 복기 페이지 (플코드)

정적 단일 파일이다. 외부 라이브러리 0, 빌드 0, 서버 0 — `review-data.json` 하나만 fetch한다. 그래서 GitHub Pages에 올리면 그대로 돈다.

[주의: 아래는 7/22에 배포한 최초 버전이다. 코드 재현의 비교 로직은 다음 날(7/23) LCS 방식으로 교체됐다 — 그 사연과 교체 코드는 7/23 파일에 있다.]

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>복기지 - backend-journey</title>
<style>
:root { --bg:#0f1115; --card:#181b22; --line:#2a2f3a; --tx:#e6e8ee; --sub:#9aa3b2;
--ok:#3fb96f; --mid:#e0b64a; --bad:#e06a5a; --acc:#5b8def; }
* { box-sizing:border-box; }
body { margin:0; background:var(--bg); color:var(--tx);
font-family:-apple-system,BlinkMacSystemFont,"Apple SD Gothic Neo","Malgun Gothic",sans-serif; }
.wrap { max-width:760px; margin:0 auto; padding:16px 14px 60px; }
h1 { font-size:20px; margin:6px 0 2px; }
.sub { color:var(--sub); font-size:13px; margin-bottom:14px; }
.tabs { display:flex; gap:8px; margin:10px 0 14px; }
.tabs button { flex:1; padding:10px 0; border:1px solid var(--line); background:var(--card);
color:var(--tx); border-radius:10px; font-size:15px; }
.tabs button.on { border-color:var(--acc); color:var(--acc); font-weight:700; }
.bar { display:flex; flex-wrap:wrap; gap:8px; margin-bottom:12px; align-items:center; }
select,input[type=number] { background:var(--card); color:var(--tx); border:1px solid var(--line);
border-radius:8px; padding:7px 8px; font-size:14px; }
```

```

label.chk { font-size:13px; color:var(--sub); display:flex; align-items:center; gap:4px; }
.card { background:var(--card); border:1px solid var(--line); border-radius:12px; padding:14px; margin-bottom:12px; }
.meta { font-size:12px; color:var(--sub); margin-bottom:8px; display:flex; justify-content:space-between; }
.q { font-size:16px; line-height:1.55; white-space:pre-wrap; }
textarea { width:100%; min-height:110px; margin-top:10px; background:#101319; color:var(--tx);
    border:1px solid var(--line); border-radius:8px; padding:10px; font-size:14px; line-height:1.5; }
textarea.code { font-family:ui-monospace,Menlo,Consolas,monospace; min-height:260px; white-space:pre; }
.row { display:flex; gap:8px; margin-top:10px; flex-wrap:wrap; }
.row button, .row a { flex:1; min-width:90px; text-align:center; padding:10px 0; border-radius:9px;
    border:1px solid var(--line); background:#12151c; color:var(--tx); font-size:14px; text-decoration:none; }
.g button.sel-0 { border-color:var(--ok); color:var(--ok); font-weight:700; }
.g button.sel-T { border-color:var(--mid); color:var(--mid); font-weight:700; }
.g button.sel-X { border-color:var(--bad); color:var(--bad); font-weight:700; }
.nav { display:flex; gap:8px; }
.nav button { flex:1; padding:11px 0; border-radius:9px; border:1px solid var(--line); background:#12151c; color:var(--tx); }
.nav .go { border-color:var(--acc); color:var(--acc); font-weight:700; }
pre.origin { background:#101319; border:1px solid var(--line); border-radius:8px; padding:10px;
    overflow-x:auto; font-size:13px; line-height:1.45; }
table.diff { width:100%; border-collapse:collapse; font-family:ui-monospace,Menlo,Consolas,monospace;
    font-size:12px; margin-top:10px; }
table.diff td { border-top:1px solid var(--line); padding:3px 6px; vertical-align:top; white-space:pre-wrap; word-break:break-all; }
td.no { color:var(--sub); width:30px; }
td.st { width:22px; }
tr.ok td.st { color:var(--ok); } tr.bad td.st { color:var(--bad); }
tr.bad { background:rgba(224,106,90,.07); }
.score { font-size:22px; font-weight:800; margin-top:10px; }
.score.hi { color:var(--ok); } .score.md { color:var(--mid); } .score.lo { color:var(--bad); }
.hint { font-size:12px; color:var(--sub); margin-top:6px; line-height:1.5; }
.timer { font-variant-numeric:tabular-nums; color:var(--sub); font-size:13px; }
.empty { color:var(--sub); text-align:center; padding:40px 0; }
#exportBox { display:none; }
</style>
</head>
<body>
<div class="wrap">
    <h1>복가지</h1>
    <div class="sub" id="dataInfo">데이터 불러오는 중...</div>

    <div class="tabs">
        <button id="tabQ" class="on" onclick="setMode('q')">문제 복기</button>
        <button id="tabC" onclick="setMode('c')">코드 재현</button>
    </div>

    <!-- ===== 문제 모드 ===== -->
    <div id="modeQ">
        <div class="bar">
            <select id="daySel" onchange="buildPool()"></select>
            <select id="orderSel" onchange="buildPool()">
                <option value="seq">순서대로</option>
                <option value="rand">랜덤</option>
            </select>
            <select id="cntSel" onchange="buildPool()">
                <option value="0">전체</option><option value="5">5문항</option><option value="10">10문항</option>
            </select>
            <label class="chk"><input type="checkbox" id="starOnly" onchange="buildPool()">★만</label>
        </div>
        <div id="qArea"></div>
        <div class="nav">
            <button onclick="move(-1)">← 이전</button>
            <button class="go" onclick="move(1)">다음 →</button>
        </div>
        <div class="row">
            <button onclick="exportAnswers()">답안 내보내기 (복사)</button>
        </div>
        <textarea id="exportBox" readonly></textarea>
        <div class="hint">사용법: 답을 직접 타이핑 → [출처 열기]로 그날 로그의 해당 섹션을 읽고 0/△/X 자가채점.
        △·X 문항은 [답안 내보내기] → 네이버 내게쓰기 → 폰에서 클로드 채팅에 붙이면 채점·꼬리질문을 받는다.</div>

```

```

</div>

<!-- ===== 코드 재현 모드 ===== -->
<div id="modeC" style="display:none">
  <div class="bar">
    <select id="codeSel" onchange="pickCode()"></select>
    <label class="chk"><input type="checkbox" id="blind" checked onchange="pickCode()">독타이핑(원본 숨김)</label>
    <span class="timer" id="timer">00:00</span>
  </div>
  <pre class="origin" id="originPre" style="display:none"></pre>
  <textarea class="code" id="codeInput" spellcheck="false"
    placeholder="여기에 기억으로 코드를 타이핑한다. 첫 키 입력에 타이머 시작."></textarea>
  <div class="row">
    <button onclick="compareCode()">비교 · 채점</button>
    <button onclick="resetCode()">다시</button>
  </div>
  <div id="diffOut"></div>
  <div class="hint">독타이핑 = 빈 화면에 기억으로(7/9 재현훈련의 상설판). 체크 해제하면 보고치기(원본 보며 타이핑).
  비교는 줄 단위 - 들어쓰기·철자까지 정확히. 끝 공백만 무시한다.</div>
</div>
</div>

<script>
let DATA=null, pool=[], idx=0, ans={}, curCode=null, t0=null, tInt=null, lastCodeResult=null;

fetch('review-data.json',{cache:'no-store'}).then(r=>r.json()).then(d=>{
  DATA=d; init();
}).catch(()=>{ document.getElementById('dataInfo').textContent='review-data.json 로드 실패 - 크론이 아직 안 돌아왔거나 경로 문제'; });

function init(){
  const days=DATA.days;
  const nq=days.reduce((s,x)=>s+x.questions.length,0);
  const nc=days.reduce((s,x)=>s+x.codes.length,0);
  document.getElementById('dataInfo').textContent=
    `최신 ${DATA.latest} · ${days.length}일치 · 문제 ${nq} · 코드 ${nc} (로그 올리면 1분 내 자동 갱신)`;
  const ds=document.getElementById('daySel');
  ds.innerHTML='';
  ds.append(new Option('최신 하루 ('+DATA.latest+')','latest'));
  ds.append(new Option('전체 누적','all'));
  [...days].reverse().forEach(x=>ds.append(new Option(x.date,x.date)));
  const cs=document.getElementById('codeSel');
  cs.innerHTML='';
  days.forEach(x=>x.codes.forEach((c,i)=>{
    cs.append(new Option(`${x.date} · ${c.title}`.slice(0,60), x.date+'|'+i));
  }));
  buildPool(); pickCode();
}

function setMode(m){
  document.getElementById('modeQ').style.display = m==='q'?':none';
  document.getElementById('modeC').style.display = m==='c'?':none';
  document.getElementById('tabQ').classList.toggle('on',m==='q');
  document.getElementById('tabC').classList.toggle('on',m==='c');
}

/* ----- 문제 모드 ----- */
function buildPool(){
  const sel=document.getElementById('daySel').value||'latest';
  const star=document.getElementById('starOnly').checked;
  const order=document.getElementById('orderSel').value;
  const cnt=+document.getElementById('cntSel').value;
  let days=DATA.days;
  if(sel==='latest') days=days.filter(x=>x.date===DATA.latest);
  else if(sel!=='all') days=days.filter(x=>x.date===sel);
  pool=[];
  days.forEach(x=>x.questions.forEach(q=>{
    if(star&&!q.star) return;

```

```

        pool.push({date:x.date,n:q.n,q:q.q,star:q.star});
    }));
    if(order===`rand`) for(let i=pool.length-1;i>0;i--){const j=Math.floor(Math.random()*(i+1));[pool[i],pool[j]]=[pool[j],pool[i]];}
    if(cnt>0) pool=pool.slice(0,cnt);
    idx=0; renderQ();
}

function key(it){ return it.date+'#'+it.n; }

function renderQ(){
    const area=document.getElementById('qArea');
    area.innerHTML='';
    if(!pool.length){ area.innerHTML=`<div class="empty">이 범위엔 문항이 없다</div>`; return; }
    const it=pool[idx], k=key(it), st=ans[k]||{a:'',g:''};
    const card=document.createElement('div'); card.className='card';
    const meta=document.createElement('div'); meta.className='meta';
    meta.innerHTML=`<span>${it.date} · #${it.n}${it.star?' ★':''}</span><span>${idx+1} / ${pool.length}</span>`;
    const q=document.createElement('div'); q.className='q';
    q.textContent=it.q.replace(/\*\*/g, '');
    const ta=document.createElement('textarea');
    ta.placeholder='답을 타이핑한다 - 쓰는 것 자체가 인출 훈련이다';
    ta.value=st.a;
    ta.oninput=()=>{ (ans[k]=ans[k]||{a:'',g:''}).a=ta.value; };
    const g=document.createElement('div'); g.className='row g';
    [['O','맞음'],['T','애매 △'],['X','틀림']].forEach(([v,label])=>{
        const b=document.createElement('button');
        b.textContent=label;
        if(st.g===v) b.classList.add('sel-'+v);
        b.onclick=()=>{ (ans[k]=ans[k]||{a:'',g:''}).g=v; renderQ(); };
        g.appendChild(b);
    });
    const src=document.createElement('div'); src.className='row';
    const a=document.createElement('a');
    a.href=`https://github.com/Taewoo-Won/backend-journey/blob/main/logs/${it.date}.md`;
    a.target='_blank'; a.textContent='출처 열기 (그날 로그)';
    src.appendChild(a);
    card.append(meta,q,ta,g,src);
    area.appendChild(card);
}

function move(d){ if(!pool.length)return; idx=Math.min(pool.length-1,Math.max(0,idx+d)); renderQ(); }

function exportAnswers(){
    const lines=['[복가지 답안] '+new Date().toISOString().slice(0,10)];
    let n=0;
    pool.forEach(it=>{
        const st=ans[key(it)];
        if(!st||(!st.a&&!st.g)) return;
        n++;
        const gtxt={O:'O',T:'△',X:'X'}[st.g]||'-';
        lines.push(`-`, `Q(${it.date} #${it.n}${it.star?' ★':''}): ${it.q.replace(/\*\*/g, '')}`, `답: ${st.a||'(빈칸)'}`, `자평: ${gtxt}`);
    });
    if(lastCodeResult) lines.push(`-`,`[코드 재현] '+lastCodeResult);
    if(n===0&&!lastCodeResult){ alert('내보낼 답안이 없다'); return; }
    const txt=lines.join('\n');
    const box=document.getElementById('exportBox');
    box.style.display='block'; box.value=txt; box.select();
    if(navigator.clipboard) navigator.clipboard.writeText(txt).then(()=>alert('복사됨 - 네이버 내게쓰기에 붙여라')).catch(()=>alert('아래 박스에서 수동 복사'));
    else { document.execCommand('copy'); alert('복사 시도됨 - 안 되면 아래 박스에서 수동 복사'); }
}

/* ----- 코드 재현 모드 ----- */

function pickCode(){
    const v=document.getElementById('codeSel').value;
    if(!v){ curCode=null; return; }
    const [date,i]=v.split('|');

```



```

const day=DATA.days.find(x=>x.date===date);
curCode=day?day.codes[+i]:null;
const pre=document.getElementById('originPre');
const blind=document.getElementById('blind').checked;
if(curCode&&!blind){ pre.style.display='block'; pre.textContent=curCode.code; }
else pre.style.display='none';
document.getElementById('diffOut').innerHTML='';
}

document.getElementById('codeInput').addEventListener('input',()=>{
  if(t0===null){ t0=Date.now();
    tInt=setInterval(()=>{ const s=Math.floor((Date.now()-t0)/1000);
      document.getElementById('timer').textContent=String(Math.floor(s/60)).padStart(2,'0')+':'+String(s%60).padStart(2,'0');
    },500);
  }
});

function resetCode(){
  document.getElementById('codeInput').value='';
  document.getElementById('diffOut').innerHTML='';
  t0=null; clearInterval(tInt); document.getElementById('timer').textContent='00:00';
}

function compareCode(){
  if(!curCode){ alert('코드를 선택해라'); return; }
  clearInterval(tInt);
  const exp=curCode.code.split('\n').map(s=>s.replace(/\s+$/, ''));
  const got=document.getElementById('codeInput').value.split('\n').map(s=>s.replace(/\s+$/, ''));
  const n=Math.max(exp.length, got.length);
  let eq=0;
  const tbl=document.createElement('table'); tbl.className='diff';
  for(let i=0; i<n; i++){
    const e=exp[i]??'', g=got[i]??'', same=(e===g);
    if(same) eq++;
    const tr=document.createElement('tr'); tr.className=same?'ok':'bad';
    const td1=document.createElement('td'); td1.className='no'; td1.textContent=i+1;
    const td2=document.createElement('td'); td2.className='st'; td2.textContent=same?'✓':'x';
    const td3=document.createElement('td');
    td3.textContent = same ? e : (g||'(빈 줄)'+'\n→' +(e||'(원본엔 없음)'));
    tr.append(td1,td2,td3); tbl.appendChild(tr);
  }
  const pct=Math.round(eq/n*100);
  const out=document.getElementById('diffOut'); out.innerHTML='';
  const sc=document.createElement('div');
  sc.className='score' +'+(pct>=90?'hi':pct>=60?'md':'lo');
  sc.textContent=`일치율 ${pct}% (${eq}/${n}줄) · ${document.getElementById('timer').textContent}`;
  out.append(sc, tbl);
  lastCodeResult=`${curCode.title} - 일치율 ${pct}% (${eq}/${n}줄), ${document.getElementById('timer').textContent},
${document.getElementById('blind').checked?'독타이핑': '보고치기'}`;
}
</script>
</body>
</html>

```

구조 해설

① 데이터 로딩 — 한 번의 fetch가 전부

```
fetch('review-data.json',{cache:'no-store'})
```

`cache:'no-store'` 가 핵심이다. 이게 없으면 브라우저가 옛 JSON을 캐시해서, 오늘 올린 작업기록의 문항이 안 보인다.

② 상태는 전부 메모리 변수

```
let DATA=null, pool=[], idx=0, ans={}, curCode=null, t0=null, tInt=null, lastCodeResult=null;
```

- `DATA` = 전체 데이터 / `pool` = 필터 적용된 현재 문제 목록 / `idx` = 지금 몇 번째
- `ans` = 답안 저장소. 키가 `"2026-07-20#5"` 꼴이라(날짜+문항번호) 필터를 바꿔도 답이 유지된다

- localStorage를 안 쓰는 이유: 사지방 PC는 공용이고 초기화된다. 어차피 세션 단위 훈련이라 메모리로 충분하다

③ 필터 조합

```
if(sel==='latest') days=days.filter(x=>x.date===DATA.latest);
else if(sel!=='all') days=days.filter(x=>x.date===sel);
```

"최신 하루 / 전체 누적 / 특정 날짜"를 한 값으로 처리한다. 그 위에 ★ 필터, 랜덤 셔플(Fisher-Yates), 개수 제한을 겹친다.

```
if(order==='rand') for(let i=pool.length-1;i>0;i--){const j=Math.floor(Math.random()*(i+1));[pool[i],pool[j]]=pool[j],pool[i];}
```

Fisher-Yates 셔플 — 뒤에서부터 앞의 임의 원소와 자리를 바꾼다. `sort(()=>Math.random()-0.5)` 같은 흔한 방법은 편향이 생기는데, 이걸 모든 순열이 균등하게 나온다.

④ 타이머는 첫 키 입력에 시작

```
document.getElementById('codeInput').addEventListener('input',()=>{
  if(t0===null){ t0=Date.now(); ... }
});
```

페이지 열자마자가 아니라 **실제로 치기 시작할 때** 시작한다. 문제를 읽고 생각하는 시간은 안 재고, 재현 속도만 잰다.

⑤ 내보내기

```
if(navigator.clipboard) navigator.clipboard.writeText(txt)...
else { document.execCommand('copy'); ... }
```

클립보드 API가 막힌 환경(구형 브라우저·비 HTTPS)을 대비해 구식 `execCommand` 로 폴백하고, 그마저 안 되면 textarea에 띄워 수동 복사하게 한다. **사지방 PC 환경을 모르니 3단 폴백.**

5. 크론 배선 — ; 와 && 의 차이

기존 크론(7/20에 정체 확인한 그 자동봇):

```
* * * * * cd /root/backend-journey && /usr/bin/python3 update.py >> /root/backend-journey/.cron.log 2>&1
```

교체한 크론:

```
* * * * * cd /root/backend-journey && /usr/bin/python3 make_review.py >> /root/backend-journey/.cron.log 2>&1; /usr/bin/python3 update.py >> /root/backend-journey/.cron.log 2>&1
```

두 스크립트를 잇는 기호가 **&&** 가 아니라 **;** 인 게 의도된 설계다:

기호	의미	이 상황에서
A && B	A가 성공해야 B 실행	make_review가 죽으면 대시보드 붓이 같이 멈춘다 — 부작용이 너무 크다
A ; B ✓	A의 결과와 무관하게 B 실행	추출기가 죽어도 대시보드는 계속 돈다

즉 **새로 붙인 기능이 기존 기능을 볼모로 잡지 않게** 한 것. 스크립트 자체도 어떤 예외든 삼키고 `exit 0` 으로 끝나게 짰으니(§3 ㉟), 이중으로 막았다.

그리고 왜 `make_review`가 먼저인가: `update.py`가 `git add .` 로 커밋하기 **전에** JSON이 갱신돼 있어야 같은 사이클에 함께 올라간다. 순서가 반대면 한 사이클(1분) 늦게 반영된다.

update.py가 어떻게 커밋하는지 확인 (배선 전에 봐야 했던 것):

```
root@ledger:~# grep -n "git\|add\|commit\|push" ~/backend-journey/update.py
26:def git(*args, capture=False):
27:     return subprocess.run(["git", *args], cwd=str(HERE),
95:# — git: 실제 변경/미푸시가 있을 때만 커밋·푸시 —
96:git("add", ".");
97:has_changes = git("diff", "--cached", "--quiet").returncode != 0
101:  git("commit", "-q", "-m", msg)
102:unpushed = git("rev-list", "origin/main..HEAD", "--count", capture=True).stdout.strip()
104:if has_changes or (unpushed and unpushed != "0"):
105:  git("push", "-q"); pushed = True
```

96줄 `git("add", ".")` — **레포에 떨어지는 새 파일은 전부 자동으로 담긴다.** 그래서 `review.html`·`make_review.py`·`review-data.json` 셋 다 별도 조치 없이 커밋된다. 97줄에서 실제 변경이 있을 때만 커밋하는 것도 확인 — §3의 "결정적 출력"이 여기서 값을 한다.

6. 배포 마찰 2건

마찰 ① — 폰이 파일명의 언더바를 공백으로 바꾼다 (★ 새 합정)

SFTP로 두 파일을 올렸는데 검증에서 이상한 결과가 나왔다:

```
root@ledger:~/backend-journey# ls -la make_review.py review.html
ls: cannot access 'make_review.py': No such file or directory
-rw-r--r-- 1 root root 13828 Jul 22 11:19 review.html
```

review.html은 올라갔는데 make_review.py만 없다. 같은 방식으로 같이 올렸는데. 한참 뒤 git 파일 목록에서 범인이 드러났다:

```
root@ledger:~/backend-journey# git ls-tree --name-only HEAD | grep review
make_review.py      ← ★ 언더바가 공백!
make_review.py
review-data.json
review.html
```

폰에서 파일을 받아 SFTP로 올리는 과정에서 make_review.py 가 make review.py 로 바뀌어 저장됐다. 업로드 자체는 성공했는데 이름이 달라서 계속 "파일 없음"으로 보인 것. review.html 은 언더바가 없어서 멀쩡했다.

교훈: 폰 경우 업로드 후엔 반드시 ls 로 실제 파일명을 확인한다. "업로드 실패"로 보이는 게 사실은 "이름 변형"일 수 있다. 그리고 앞으로 VPS에 올릴 파일명은 언더바를 피하는 게 안전하다.

정리:

```
root@ledger:~/backend-journey# rm -f "make review.py" && ls | grep review
make_review.py
review-data.json
review.html
```

마찰 ② — base64 한 줄이 폰에서 잘린다

SFTP가 이름을 망가뜨리니 우회로 base64 한 줄 배포(piano 프로젝트에서 정착시킨 방식)를 썼는데, 2376자가 폰 붙여넣기에서 중간에 잘렸다:

```
root@ledger:~/backend-journey# echo 'H4sICJWkYGoCA21ha2VfcmV2aWV3LnB5AI1X3W...' | base64 -d | gunzip > make_review.py && wc -c make_review.py -d | gunzip >
base64: invalid input
gzip: stdin: unexpected end of file
```

끝부분에 wc -c make_review.py -d | gunzip > 라는 이상한 조각이 섞였다 — 붙여넣기가 잘리면서 명령 자체가 뒤엎킨 것. 그 결과 깨진 파일이 생겼고:

```
root@ledger:~/backend-journey# python3 make_review.py
File "/root/backend-journey/make_review.py", line 7
    2) 결정적 출력 (타임스탬프)
SyntaxError: source code cannot contain null bytes
```

해결 — 두 단계: 1. 주석·공백을 걷어낸 압축판을 만들어 base64를 2376자 → 1072자로 줄임(§ 3의 두 번째 코드) 2. 그것도 한 번에 안 들어갈 수 있으니 400자씩 3조각으로 나눠 이어붙임

```
root@ledger:~/backend-journey# echo -n
'H4sICE6qYGoCA21yX21pbi5weQB9lL+07DQUxvt5CstDYc96zN0FIzTFogLdBgkJupl1j2dnxJ7MTH2Z3V3pVok0ihoIAWGioqGL6GkrsPwXEy0/8QpEji+Ms5n38+x65pQ0zkNQvAohoBdzDpIqhIa1069q9Im6UfI7DyUuVH2y1qlxtVysuo4VQ31rGZauj9WlibEWmVmeNEYXE4JXo6KVYHUs1yzSr5JLtS2WMGPyjL9DBSpx/jN0QiRHwul0CWcnoiFgUo28iaFv2TmXkKJrGSeuIg2xNVhC6aDqQC2uxrcTB90pnP39+zdPv/xMnn797u1vf8jZaQqM1jWj/XyBasyia6T1hvHiav89HUdm2f0beb44FXAQcBsUUGEKfgUkNK5jYnwf' > /tmp/b.txt
root@ledger:~/backend-journey# echo -n
'rEIXAnTE+cyhct7ouh6iLs0ZX8o9jW3gvb9hlap7hnHZgdRtm90+UE8L5xPzXNC0Fp2g0TEt6F8/fktzru5xdFjueGUj5YENL2xEH9fX16/1rV56Jmfcxw9M/viwEkZjFXlfnviaIkuPT0CkXWyDD06G+b0zj96vyiDT873didcgzpGMX24E089HtbFoih3KE94LwcUhqqjxRoW8/Orw6JZw7Zqnv784e33P1FBszVa5PuWTLSpj548UKMTzhjxHC0h0t5CcsEDooXxX3wt4XHohEY7z7Z0zBHfNvMFbCxrGHv5Lq3DDVAub+rwitGLFxcFzGeyMZQfoE3xvjC7JeUma7H/tFklU0nMeszt/I2ifarmH1Iuwlymbfieh92Utk3kk+GBno95B4U2' >> /tmp/b.txt
root@ledger:~/backend-journey# echo -n
'or119m60C9UyHwpjK6WnyGNp+qYFhFkjB0hIYqC5GLlcZZrmmSZyRIeQNY/CeuijXWkonV0fahQIsHhg6BQiKIZagQXJReax+treg/oy9nbXiUHajYME7ATGUfkPuv/BoVTaFON0/jeQVgOMK25aFeSd5+sV9FXlNowOPCR0UH6Jd3kXsS/GZGkj/pXvMgDaaWtdWoZyEfgkex4rYnKamWgEV7QxdyptIrExFLRYkc9ZhUcybqXBb3yS35FGYi/45B/faU02vQUAAA==' >> /tmp/b.txt
root@ledger:~/backend-journey# cd ~/backend-journey && base64 -d /tmp/b.txt | gunzip > make_review.py && python3 make_review.py && python3 -c
"import json;d=json.load(open('review-data.json'));print('days:',len(d['days']),'문항:',sum(len(x['questions'])for x in d['days']),'코드:',sum(len(x['codes'])for x in d['days']))"
days: 32 문항: 163 코드: 57
```

왜 이 방식이 안전한가: - echo -n — 줄바꿈을 안 붙인다. base64 문자열 중간에 개행이 섞이면 디코드가 깨진다. - 첫 조각은 > (새로 만들기), 나머지는 >> (이어붙이기) — 중간에 실패하면 첫 조각부터 다시 하면 되니 복구가 쉽다. - 마지막에 한 번에 base64 -d | gunzip — 조립과 검증이 한 명령에서 끝난다.

7. 검증 — 실제로 살아있는가

① 데이터 추출 결과

```
days: 32 문항: 163 코드: 57
```

32일치 로그에서 검증 질문 163개와 java 코드블록 57개를 수거했다.

② 크론이 자동 커밋했는지

```
root@ledger:~/backend-journey# git status --short; echo "--- HEAD 파일 ---"; git ls-tree --name-only HEAD | grep review
--- HEAD 파일 ---
make_review.py
review-data.json
review.html
```

`git status --short` 가 빈 출력 = 커밋 안 된 파일 없음. HEAD에 세 파일 다 있음 = 푸시 완료.

③ GitHub Pages가 실제로 서빙하는지 (브라우저 없이 curl로 판정)

```
root@ledger:~/backend-journey# for f in review.html review-data.json; do echo -n "$f: "; curl -s -o /dev/null -w "%{http_code}\n" https://taewoo-won.github.io/backend-journey/$f; done
review.html: 200
review-data.json: 200
```

④ 내용까지 확인

```
root@ledger:~/backend-journey# curl -s https://taewoo-won.github.io/backend-journey/review-data.json | head -c 120; echo
{"days":[{"codes":[{"code":"    public class Hello {\n                public static void main(String[] args) {\n                                System.
```

JSON 구조가 정상이고 실제 코드가 들어 있다(첫 항목은 6월 초 Hello World).

⑤ 잔디

```
root@ledger:~/backend-journey# git log --oneline -1
9f6149c (HEAD -> main, origin/main) update: 2026-07-22
```

8. 문제은행 현황과 작동 방식

흐름 한 줄: 작업기록을 `logs/` 에 올린다 → 1분 내 크론이 `make_review.py` 로 문항·코드를 수거 → `review-data.json` 갱신 → `update.py` 가 자동 커밋·push → GitHub Pages 반영 → 브라우저에서 보기.

누적 방식: 매번 32일치 전체를 다시 읽어 통째로 재빌드한다. 그래서 ① 새 기록을 올리면 그날 문항이 자동 추가되고 ② 옛날 로그를 고치면 그 수정도 바로 반영된다. 내용이 안 바뀌면 파일을 안 건드려니 커밋 스팸도 없다.

분포: 하루 평균 약 5문항이지만 이건 평균의 착시다. 초기 6월 로그엔 "검증 질문" 형식 자체가 없어 **0문항인 날이 여럿** 있고, 형식이 자리 잡은 뒤로는 그날 밀도에 비례해 **3~12개** 범위다(목직한 날 9~12 / 보통 5~7 / 가벼운 날 3~4). 코드블록도 마찬가지 — 초기 로그엔 풀코드를 많이 실어 많고, 7/20처럼 코드가 적은 날은 2개다.

9. 로드맵 위치

- **오늘 것은 학습 인프라** — 로드맵 마일스톤이 아니라 매일의 수율을 올리는 도구. 다만 만들면서 나온 것들(결정적 출력, 원자적 교체, `;` vs `&&`, 정규식 비탐욕 매칭)은 백엔드 소재로 그대로 쓸모가 있다.
- **밀린 복기 큐(13일+):** 7/5 원자성 트레이스 + 문답집 ★15문항. **이제 이걸 복기지에서 돌릴 수 있다** — 문항은 이미 163개가 들어 있다.
- m101~m106 done, m107(월말 점검)만 남음. codecrafters-redis Stage 2(PING/PONG)는 자유 슬롯.
- 8월 WAL 이유 3개 유지: 원자성 한계(7/5) · 영속성 부재(7/12) · 롤백 불가(7/13).

10. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. 폰 워크플로우에서 수율이 떨어진 원인을 "화면이 작아서"가 아니라 무엇으로 진단했나? 손의 역할이 어떻게 달라졌나?
2. 문제를 "생성"이 아니라 "수거"로 설계한 근거는? 이미 매일 생산되고 있던 네 가지 자산은?
3. AI를 페이지 안에 직접 넣지 않은 이유는? 대신 택한 두 경로(②③)는 각각 무엇이고 언제 쓰나?
4. `re.M` 과 `re.S` 는 각각 무엇을 바꾸나? `.*?` (비탐욕)를 안 쓰고 `.*` 를 쓰면 코드블록 추출이 어떻게 망가지나?
5. `review-data.json`에 타임스탬프를 넣지 않은 이유는? `sort_keys=True` 가 왜 필요한가?
6. 임시 파일에 쓰고 `os.replace` 로 갈아끼우는 이유는? 그냥 덮어쓰면 무슨 일이 생길 수 있나?
7. 크론에서 두 스크립트를 `&&` 가 아니라 `;` 로 이은 이유는? `make_review`가 `update.py`보다 먼저 와야 하는 이유는?
8. `make_review.py`가 어떤 예외에도 `exit 0` 으로 끝나게 만든 이유는? 크론 배선의 어떤 선택과 짝을 이루나?
9. Fisher-Yates 셔플을 쓴 이유는? `sort()==>Math.random()-0.5` 의 문제는?
0. `fetch`에 `cache: 'no-store'` 를 준 이유는? 없으면 어떤 증상이 나타나나?
1. SFTP 업로드가 "성공했는데 파일이 없어 보인" 진짜 원인은? 앞으로의 확인 절차는?

2. base64 한 줄 배포가 폰에서 실패했을 때, 길이를 줄인 방법과 나눠 붙인 방법 두 가지는? `echo -n` 의 `-n` 이 왜 중요한가?

11. 회고

오늘 만든 건 코드를 배우는 일이 아니라 배우는 방식 자체를 고치는 일이었다. 시작은 불편한 자각이었다 — IDEA로 하던 때보다 남는 게 적다. 그걸 "폰이라 어쩔 수 없다"로 덮지 않고 원인을 쫓겐 게 오늘의 출발이었고, 답은 화면 크기가 아니라 손의 역할이었다. 예전엔 손이 코드를 생성했는데 지금은 받은 걸 옮긴다. 인출과 생성이 빠지면 인코딩이 알아지는 건 당연하다. 증거도 이미 쌓여 있었다 — 내가 직접 내린 결정의 이름(InvalidArgumentException)이 안 떠오르고, 두 번째 쓰는 record에서 필드를 빠뜨린 것.

가장 좋았던 순간은 인프라가 이미 서 있다는 걸 깨달았을 때다. 매일 파일 끝에 박아온 "검증 질문"이 그대로 문제은행이고, C축 원칙으로 실어온 풀코드가 그대로 독타이핑 원본이었다. 공개 Pages도, 매분 도는 크론도 이미 있었다. 그래서 AI에게 문제를 지어내게 할 필요가 없었다 — 수거만 하면 됐다. 이게 결과적으로 더 좋은 설계이기도 하다. 남이 만든 일반적인 문제가 아니라, 그날 내가 "이건 물어봐야 한다"고 판단해 남긴 문항이 그대로 나오니까. 문제의 질이 곧 작업기록의 질이고, 그건 이미 관리되고 있다.

AI를 어디에 돌지가 제일 오래 고민한 지점이었다. "문제 푸는 과정에 AI가 안 들어가는데 지장 없냐"는 물음이 정확했고, 답은 "모드마다 다르다"였다. 코드 재현은 기계 대조가 오히려 정확하니 AI가 필요 없고, 서술형은 쓰는 것 자체가 이미 인출 훈련이라 약한 고리는 채점뿐이다. 그래서 페이지엔 안 넣고(공개·비로그인이라 키를 못 박는다) 답안 내보 내기로 비동기 채점을 붙였다. AI가 빠진 게 아니라 비동기로 들어간 것이고, 그쪽이 7주 이력을 다 아는 상태로 채점하니 오히려 낫다.

배포에서 두 번 걸렸다. 하나는 폰이 파일명의 언더바를 공백으로 바꿔놓은 것 — 업로드는 성공했는데 이름이 달라 "없는 파일"로 보였고, git 파일 목록을 보고서야 `make review.py` 가 눈에 들어왔다. 이걸 앞으로도 계속 물릴 함정이라 확인 절차로 못 박았다. 다른 하나는 base64 한 줄이 폰에서 잘린 것. 주석을 건너내 길이를 반으로 줄이고 400 자씩 세 조각으로 나눠 조립하니 통과했다. 둘 다 "폰으로만 개발한다"는 제약에서 나오는 종류의 문제고, 하나씩 절차로 굳히는 수밖에 없다.

이걸로 사지방 시간이 복기 시간이 된다. 지금 163문항 57코드가 들어 있고, 13일 밀린 복기 큐도 이 페이지에서 돌릴 수 있다. 다만 하나 남았다 — 코드 비교가 지금은 줄 번호를 맞대는 순진한 방식이라, 중간에 한 줄만 빠뜨려도 아래가 통째로 밀려 오답으로 잡힌다. 실제로는 95% 맞게 쳤는데 20%가 틀 수 있다는 뜻이고, 학습 도구로는 그게 사람 김을 뺀다. 그건 내일 고친다.

한 줄: "폰이라 머리에 덜 남는다"의 원인을 손의 역할(생성→복사)로 진단하고, 매일 쓰는 작업기록이 스스로 문제은행이 되는 복기지를 만들었다 — 문제는 생성이 아니라 수거, AI는 페이지 밖 비동기로. 163문항·57코드 적재, Pages 200 확인.